

A Decentralized Drone Swarm Controller for Autonomous Search and Rescue

Technical analysis of four key subsystems

Étienne Pélissier

Team *Anduril* (#119), *Swarm-Rescue* challenge

Abstract

Swarm-Rescue asks each team to program the on-board intelligence of a fleet of ten fully autonomous drones. The drones must explore an unknown, GPS-degraded indoor environment, locate wounded people, carry them back to a rescue center, and survive. This report dissects the controller I built for the challenge. The full source is available at <https://github.com/npaincomplet/swarm-rescu3>; the simulation environment was provided by the organizers, and my own contribution lives entirely under `src/swarm_rescue/solutions`. Rather than surveying every line of code, the report focuses on four subsystems that carry most of the engineering and scientific weight. The first is a probabilistic occupancy-mapping scheme built on a grid of accumulated evidence, extended to a complex-valued grid that separates trustworthy from noisy observations. The second is a decentralized frontier-based exploration strategy whose target assignment is solved as a linear assignment problem. The third is a two-layer motion stack that combines global A* planning on an inflated cost map with a reactive obstacle-avoidance controller. The fourth is a set of communication mechanisms designed to keep the swarm coherent under uncertainty, propagating persistent information across drones that never meet and disambiguating noisy, repeated observations of the same object. For each subsystem I state the problem and its constraints, derive the relevant equations, and discuss the design rationale and its trade-offs. Every parameter value cited is traceable to the controller's configuration module.

1 Introduction and system overview

1.1 The mission and how it is scored

The mission is deliberately reminiscent of a collapsed-building search. A fleet of ten identical drones starts at a rescue center and must explore an a-priori-unknown 2D map, detect injured people, grasp them, and bring them back. A run ends when a time budget expires or the operators stop it.

The final score aggregates several criteria that frequently pull in different directions. The dominant reward is the number of **rescued people** delivered to the rescue center. **Exploration** adds the fraction of the map that has been observed. **Efficiency** rewards the time remaining once the map is fully explored and everyone is rescued, so fast solutions score higher than merely correct ones. **Drone health** rewards survival: drones lose life points on every collision and are destroyed at zero, and surviving drones score points.

These criteria turn the problem into a multi-objective one. Exploring aggressively conflicts with preserving health. Rushing to a wounded person conflicts with mapping carefully. And because the evaluation maps are unknown, the controller must be robust rather than tuned to a particular layout. The design described here treats rescue throughput under partial information as the primary objective, and uses exploration as the means to discover rescue opportunities.

1.2 The simulation environment

The controller runs inside a 2D simulator built on the *Simple-Playgrounds* library, which uses the *Pymunk* physics engine and the *Arcade* game engine. Three properties of this environment shape the whole design.

First, time is *discrete*. The simulator runs a perception-action-communication loop once per timestep, at roughly thirty timesteps per second. At each step the controller receives fresh sensor readings and any messages from teammates, and must return an actuator command. Everything described in this report happens inside that single per-step budget.

Second, the world is a *rectangle*. A map has a size [width, height] measured in pixels, with a standard Cartesian frame whose origin sits at the center of the map, the x-axis pointing right and the y-axis pointing up. Orientation is an angle in $[-\pi, \pi]$, zero meaning the drone faces the positive x-axis. The rectangular, bounded world is what makes a fixed-size occupancy grid a natural representation (Section 2).

Third, drones are *physical bodies*. They collide with walls and with each other, every collision costs health, and a destroyed drone simply disappears. Rescuing requires physically grasping a wounded person and carrying them, during which the carried person becomes transparent to the carrier’s sensors. Communication is local: a drone exchanges messages only with teammates within 250 pixels, and messages are sent and received once per timestep. This last fact matters for coordination, because a drone never has a global, instantaneous view of the swarm.

1.3 What the drone perceives

The simulator gives each drone a modest but realistic sensor suite, summarized in Table 1. The single most important fact about the problem is visible in the last column: almost nothing the drone perceives can be taken at face value. The noise is not even always white. The GPS and compass are corrupted by a temporally correlated, first-order autoregressive (AR(1)) process, and absolute localization disappears entirely inside designated no-GPS zones, where only the relative odometer remains. Every subsystem below is, in part, a strategy for coping with this uncertainty.

Sensor	Returns	Max range	Noise model (as modeled by the controller)
Lidar	ranges on a fan of rays	300 px	Gaussian, $\sigma \approx 2.5$ px
Semantic	type, range, bearing of entities	200 px	Gaussian on range (no wall detection)
GPS	absolute position (x, y)	n/a	AR(1), $\sigma \approx 5$ px, $\alpha = 0.98$; absent in no-GPS zones
Compass	absolute heading θ	n/a	AR(1), $\sigma \approx 4^\circ$, $\alpha = 0.98$; absent in no-GPS zones
Odometer	relative motion $[d, \alpha, \Delta\theta]$	n/a	Gaussian per component ($\sigma_d \approx 0.2$, $\sigma_\alpha \approx 8^\circ$, $\sigma_{\Delta\theta} \approx 1^\circ$)

Table 1: The drone’s inputs and their uncertainty. The semantic sensor sees wounded people, the rescue center and other drones, but not walls; walls must be inferred from lidar. The standard deviations shown are the values the controller assumes for its filtering.

1.4 Estimating the pose

Because most of the controller reasons in world coordinates, it needs a continuous estimate of its own pose from the intermittent, correlated-noise sensors of Table 1. The active estimator is an **augmented extended Kalman filter** (EKF). Its design is shaped directly by the fact that the GPS and compass noise is AR(1) rather than white.

The filter carries a six-dimensional state that augments the pose with the current value of

the sensor noise itself:

$$\mathbf{x} = [p_x, p_y, \theta, n_{gx}, n_{gy}, n_c]^\top, \quad (1)$$

where (p_x, p_y, θ) is the pose and (n_{gx}, n_{gy}, n_c) are the additive AR(1) noise terms currently affecting the two GPS axes and the compass. Modeling the noise as part of the state is the key idea. Because the noise is correlated in time, the filter can estimate its slowly-varying value and subtract it, instead of being forced to treat each GPS reading as an independent white-noise sample.

Prediction. Between measurements the pose is propagated by dead reckoning from the odometer, and the noise states decay geometrically toward zero according to their AR(1) dynamics:

$$\mathbf{p}' = \mathbf{p} + d \begin{bmatrix} \cos(\theta + \alpha) \\ \sin(\theta + \alpha) \end{bmatrix}, \quad \theta' = \theta + \Delta\theta, \quad n' = \alpha_{\text{ar}} n, \quad (2)$$

with $\alpha_{\text{ar}} = 0.98$. The AR(1) innovation variance is set to $q = (1 - \alpha_{\text{ar}}^2) \sigma_n^2$ so that the stationary variance of each noise state matches the sensor's known noise variance σ_n^2 . The covariance is propagated in the usual EKF form $P' = F P F^\top + L Q L^\top$, where F is the Jacobian of the motion model and L maps the odometer noise and the AR(1) innovations into the state.

Correction. The measurement model says that each absolute reading equals the true value plus its current noise state:

$$h(\mathbf{x}) = [p_x + n_{gx}, p_y + n_{gy}, \theta + n_c]^\top. \quad (3)$$

This is what lets the filter attribute a persistent offset to the noise term rather than to the pose. Rows are included only for the sensors that are actually available this step, the compass residual is wrapped to $[-\pi, \pi]$, and the standard Kalman gain then corrects the state.

Behavior in no-GPS zones. When GPS and compass vanish, their measurement rows simply drop out. The filter runs on prediction alone, dead-reckoning from the odometer, and its covariance grows to reflect the accumulating drift. The mapping subsystem reads a single bit from all of this, namely whether absolute positioning was available this step, and uses it to decide how much to trust what it just saw (Section 2).

1.5 The control loop

Every timestep the controller runs the same fixed sequence of operations, summarized in Algorithm 1. Perception feeds a world model (the local map plus what is known of the swarm), the state machine selects a behavior, and that behavior drives the planning and control stack to produce one command.

The behavior in step 7 is chosen by a finite-state machine. The drone is therefore *stateful*: it is always in exactly one well-defined mode (exploring, grasping a wounded person, returning to the rescue center, avoiding an obstacle, and so on), and transitions between modes are driven by explicit conditions. This statefulness is a deliberate engineering choice. It makes the controller's behavior deterministic and easy to reason about, and it makes debugging tractable, because at any instant the drone's intent is a single named state rather than an emergent blend of behaviors.

1.6 Source layout

The contributed code is organized into focused components rather than a single monolithic class. Table 2 lists the main files. One choice deserves emphasis: every tunable constant lives in one configuration module as typed dataclasses, which is why every threshold, gain and radius in this report can be quoted with a definite value and traced to a single place.

Localisation

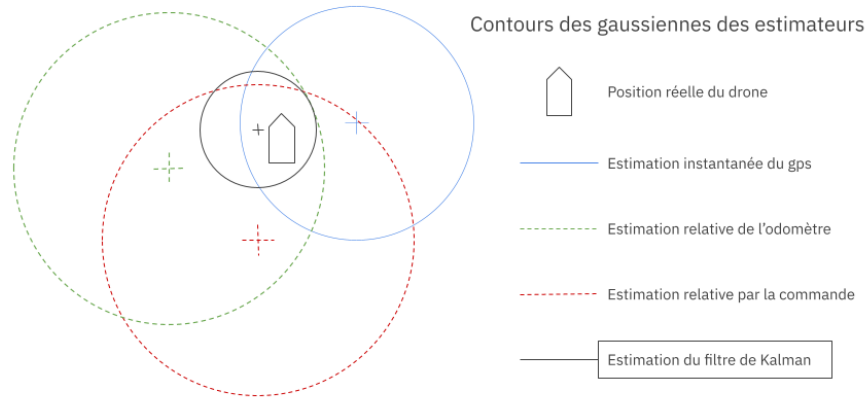


Figure 1: The competing pose estimators and their uncertainty. Each dashed contour is the Gaussian spread of one source, the instantaneous GPS, the relative odometer and the command-based dead reckoning, and the augmented EKF fuses them into the estimate centered on the drone's true position. (Slide from the team presentation; labels in French.)

Algorithm 1: One control step (the drone's `control()` method)

- 1 read raw sensors: lidar, semantic, GPS, compass, odometer
 - 2 pre-process them into situation flags (near wall, wounded in sight, near a peer, ...)
 - 3 update the EKF pose estimate (Section 1.4)
 - 4 update the occupancy grid from the lidar (Section 2)
 - 5 send messages to the swarm and merge those received (Section 5)
 - 6 evaluate transition conditions and step the state machine
 - 7 execute the active state to obtain a command
 - 8 clip the command to $[-1, 1]$ and record it in memory
 - 9 **return** command
-

Machine à états

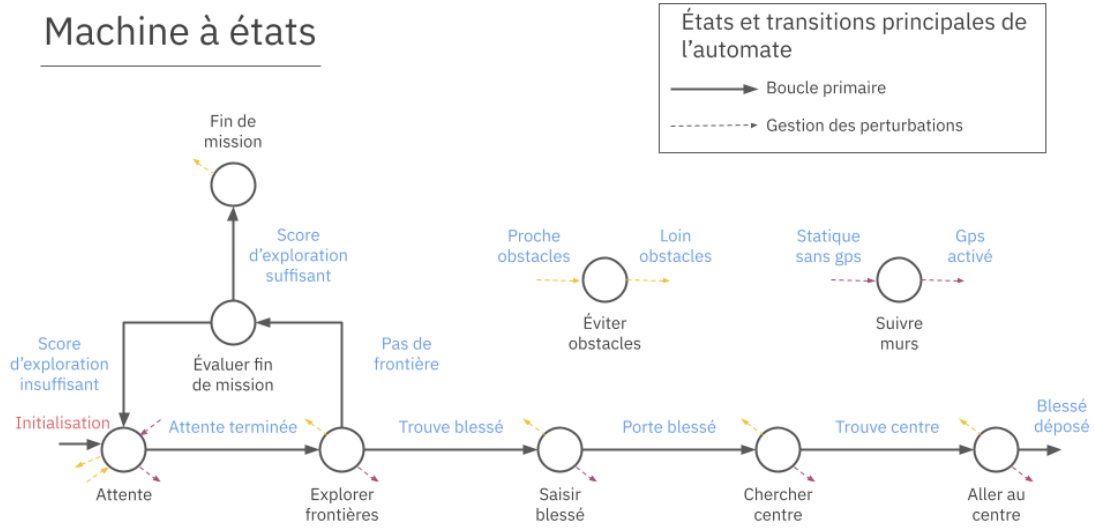


Figure 2: The finite-state machine. The solid arrows form the primary mission loop (initialize, wait, explore frontiers, grasp a wounded person, seek and return to the rescue center), while the dashed branches handle disturbances such as obstacle avoidance and wall-following when GPS is lost. (Slide from the team presentation; labels in French.)

File (under solutions/)	Responsibility
<code>my_drone_frontier_exploration.py</code>	main drone class and the per-step <code>control()</code> loop
<code>components/grids.py</code>	occupancy grid, frontier detection, ray casting
<code>components/astar.py</code>	A* search and the octile heuristic
<code>components/path_planner.py</code>	cost map, obstacle inflation, path simplification
<code>components/path_controller.py</code>	pure-pursuit path following and reactive offsets
<code>components/pid_controller.py</code>	generic PID controller
<code>components/pose.py</code>	pose estimators (the active augmented EKF)
<code>components/sensor_manager.py</code>	turns raw sensor data into situation flags
<code>components/communication.py</code>	message types, broadcasting, merging
<code>components/exploration_tracker.py</code>	bookkeeping of wounded and kill-zone sightings
<code>components/state_machine.py</code>	states and the transition table
<code>components/state_handlers.py</code>	per-state behavior
<code>components/memory.py</code>	short histories (motion, health)
<code>utils/dataclasses_config.py</code>	all tunable parameters
<code>utils/utils.py</code>	shared helpers (Bresenham, ...)

Table 2: The main source files and what each is responsible for.

2 Probabilistic occupancy mapping

Every decision the drone makes rests on its map, so the mapping subsystem is the foundation of the whole controller. This section builds it up from a single idea, a grid of accumulated evidence, and then extends that idea twice. The first extension reads the evidence as a discrete map. The second makes it robust to the loss of GPS.

2.1 Problem and constraints

The drone must build, online and from scratch, a map usable both for planning (where can I go?) and for exploration (what have I not seen yet?). Three constraints shape the design. The lidar is noisy, so a single reading should never be trusted as ground truth, and evidence must instead accumulate. The same map is shared and merged across ten drones, some of which mapped while GPS-localized and others while dead-reckoning, so the representation must carry a notion of how trustworthy each piece of evidence is. And the update runs every timestep on every drone, so it must be cheap.

2.2 An occupancy grid of accumulated evidence

The core idea is simple. The rectangular world is divided into square cells, and each cell holds a single scalar value that summarizes the evidence gathered so far about that patch of the world. A *high* value means the patch is obstructed, by a wall, another drone or a hazard. A *low* value means it is free. A value of exactly zero means nothing has ever been observed there.

Evidence is accumulated additively. Each time a cell is observed to be free it receives a fixed negative increment, each time it is observed to be occupied it receives a fixed positive increment, and the running sum is clamped to a bounded interval:

$$b_t(\mathbf{c}) = \text{clip}(b_{t-1}(\mathbf{c}) + \Delta, -40, +40), \quad \Delta \in \{\text{empty_zone_value}, \text{obstacle_zone_value}\}, \quad (4)$$

with $\text{empty_zone_value} = -0.602$ and $\text{obstacle_zone_value} = +2.0$.

This is a pragmatic analogue of a log-odds occupancy grid. A textbook log-odds grid stores $\ell = \log \frac{P(\text{occ})}{1-P(\text{occ})}$ and adds, on each observation, the log-odds of the inverse sensor model. Equation (4) is exactly this scheme, with the two log-likelihood ratios replaced by tuned constants. The numbers are not arbitrary. Note that $-0.602 \approx \log_{10} \frac{1}{4}$, so a passed-through ray contributes the log-odds of a one-to-four free-versus-occupied likelihood. The obstacle increment is larger in magnitude, which encodes that a single lidar return is much stronger evidence of an obstacle than one pass-through is of free space. That asymmetry matters because each free cell tends to be crossed by many rays while each obstacle cell is hit by only a few. Clamping the sum to a bounded interval keeps the belief responsive: a long held conviction can still be overturned by enough contrary evidence, so the map can forget a wall that was actually a transient obstacle.

2.3 From scalar evidence to a ternary map

The raw grid of Section 2.2 is a field of real-valued evidence. Most consumers, however, do not care about the magnitude of the evidence, only about its verdict. The map therefore exposes a derived *ternary* view in which the scalar of every cell is thresholded at zero into one of three labels:

$$M(\mathbf{c}) = \begin{cases} \text{OBSTACLE} & b(\mathbf{c}) > 0, \\ \text{FREE} & b(\mathbf{c}) < 0, \\ \text{UNDISCOVERED} & b(\mathbf{c}) = 0. \end{cases} \quad (5)$$

The distinction between the two representations is worth keeping clear. The continuous grid is the filter’s running belief and is what accumulates and merges. The ternary map is a cheap,

discrete snapshot derived from it on demand, and it is what frontier detection and path planning actually consume. Note that the `UNDISCOVERED` label is precisely the zero state: a cell is unknown exactly when no evidence of either kind has ever reached it, which makes "what is still unexplored" trivial to compute.

2.4 Grid geometry and the world-to-grid transform

The world is discretized at a resolution of `resolution = 8` pixels per cell. This is a deliberate eight-to-one downsampling that shrinks the grid, and therefore every per-cell operation, by a factor of sixty-four relative to pixel resolution, at the cost of geometric detail finer than a drone radius. Throughout the code a *position* is a world coordinate in pixels and a *cell* is an integer grid index. A world position $\mathbf{p}$ maps to a cell $\mathbf{c}$, and back, by an affine relation with a vertical flip:

$$\mathbf{c} = \left\lfloor \frac{1}{r} \left(\begin{bmatrix} p_x \\ -p_y \end{bmatrix} + \frac{1}{2} \mathbf{S} \right) \right\rfloor, \quad \mathbf{p} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \left(-\frac{1}{2} \mathbf{S} + r \mathbf{c} \right), \quad (6)$$

where r is the resolution and $\mathbf{S}$ the world size. The vertical flip reconciles the mathematical convention, where y points up, with image-array indexing, where the row index grows downward. Both directions are vectorized so they accept either a single point or a whole batch of points, and the rest of the code never hand-writes a coordinate conversion.

2.5 Two channels of confidence: the complex-valued grid

So far a cell holds one scalar. The refinement that makes the map robust to GPS loss is to let each cell hold *two* scalars, packed as a single complex number. The real part accumulates evidence gathered while the drone had GPS, which is high-quality because the pose was well localized. The imaginary part accumulates evidence gathered without GPS, which is noisy because it rests on a drifting dead-reckoned pose. Routing an update to the right channel costs nothing: the increments of Equation (4) are simply multiplied by the imaginary unit when GPS is unavailable.

Reading the map collapses the two channels with a precedence rule. GPS evidence is always preferred, and noisy evidence is consulted only where no GPS evidence exists:

$$\text{eff}(z) = \begin{cases} \Re(z) & \Re(z) \neq 0, \\ \Im(z) & \text{otherwise.} \end{cases} \quad (7)$$

Encoding the two confidence channels as the real and imaginary parts of one array, rather than as two separate real arrays, is more than a notational trick. It keeps the channels algebraically separate through the swarm merge, which stays a single elementwise operation (Section 5). It is also more memory and compute efficient than maintaining two parallel grids, since one complex array is allocated, indexed and updated once instead of twice. That efficiency is not a detail in this challenge. A map can easily exceed a thousand cells on a side, and the same operations run for ten drones at roughly thirty steps per second, so halving the number of array passes is a meaningful saving.

2.6 Noise-aware ray casting

Each lidar ray contributes two pieces of evidence. The cells along the ray are free, and the cell at the ray's end is usually an obstacle. Both must be tempered by the sensor's noise, whose scale is $\sigma = \text{lidar_noise_std} = 2.5$.

For free space, marking cells free all the way out to the measured range would, on a noisy long reading, paint an obstacle cell as free. The controller therefore stops the free-marking short

Cartographie

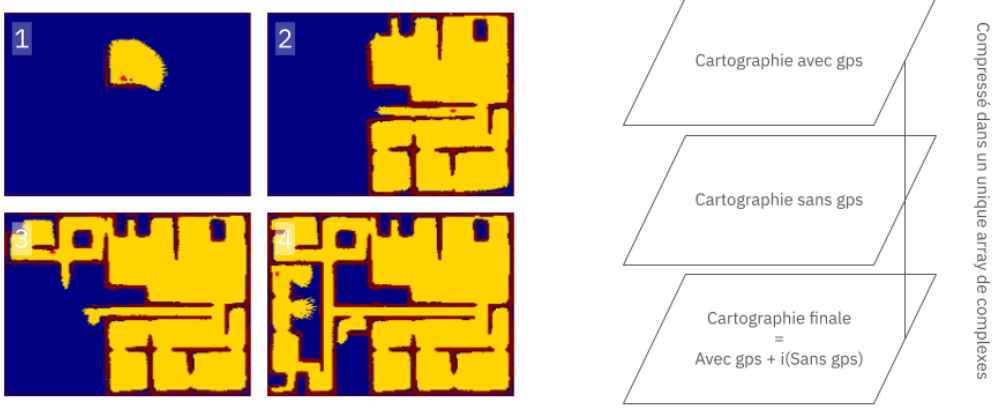


Figure 3: The two confidence channels packed into one complex grid. Evidence mapped while GPS-localized forms the real part and evidence mapped without GPS the imaginary part, so the final map is (with GPS) + i (without GPS). The four panels show the map filling in over the course of a run. (Slide from the team presentation; labels in French.)

of the measured range by an amount that grows with range:

$$d_{\text{free}} = \max\left(0, d - \text{max_lidar_dist_clip} \cdot \frac{d}{d_{\text{max}}}\right). \quad (8)$$

Long, less reliable rays are thus trusted less than short ones. For obstacles, a return is accepted as a genuine obstacle only if it is comfortably shorter than the maximum range, $d < d_{\text{max}} - \text{lidar_obstacle_margin}$, where the margin equals 4σ . This guard band rejects no-echo readings that merely saturate at maximum range. Figure 4 illustrates the geometry, and Algorithm 2 gives the full per-step procedure.

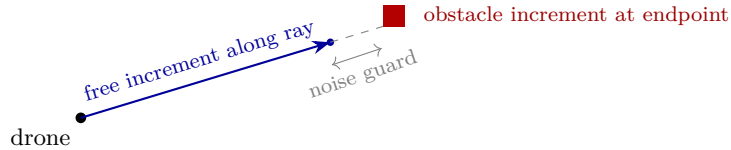


Figure 4: Noise-aware integration of one lidar ray. Free evidence is written only up to a range-dependent confidence cut, leaving a guard band before the obstacle endpoint so that noisy long returns do not erase a wall.

2.7 Implementation and performance

Cells along a ray are enumerated with the integer-only **Bresenham** line algorithm, which avoids floating-point rasterization. A subtle but important optimization is that the clamping in Equation (4) happens inside the per-update step, touching only the handful of cells just modified, rather than as a global pass over the whole grid afterwards. The cost of keeping the map bounded therefore scales with the number of cells observed this step, not with the size of the map.

Algorithm 2: Occupancy update for one drone, one timestep

Input: pose $(\mathbf{p}, \theta)$, lidar ranges $\{d_k\}$ and angles $\{\alpha_k\}$, GPS flag g

- 1 $\Delta_f \leftarrow \text{empty_zone_value}$; $\Delta_o \leftarrow \text{obstacle_zone_value}$
 - 2 **if** $\neg g$ **then** $\Delta_f \leftarrow j \Delta_f$; $\Delta_o \leftarrow j \Delta_o$ // route to the noisy channel
 - 3 **foreach** sampled ray k (every $\text{every_n} = 3\text{rd}$) **do**
 - 4 $d_{\text{free}} \leftarrow \max(0, d_k - \text{max_lidar_dist_clip} \cdot d_k / d_{\text{max}})$
 - 5 add Δ_f to the cells on the Bresenham segment from $\mathbf{p}$ outward to range d_{free}
 - 6 collect endpoints with $d_k < d_{\text{max}} - \text{lidar_obstacle_margin}$ and add Δ_o to each
 - 7 clamp every touched cell's real and imaginary parts to $[-40, 40]$
-

Cartographie

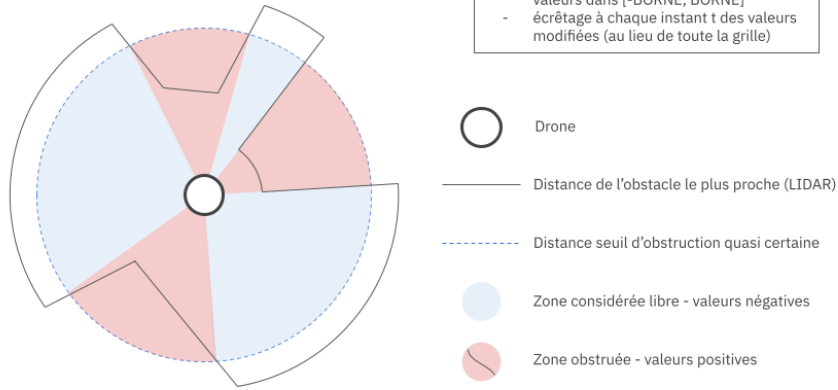


Figure 5: Ray casting into the evidence grid. Each lidar return marks the cells it crosses as free (blue, negative increment) and the cell at its endpoint as obstructed (red, positive increment), with free-marking stopped at a range-dependent confidence threshold (dashed circle). (Slide from the team presentation; labels in French.)

Concretely, with R sampled rays of maximum length L cells, one update costs $O(RL)$ for the Bresenham passes and $O(R)$ for the obstacle endpoints, entirely independent of the total cell count. Ray sub-sampling and the eight-to-one resolution are the two knobs that keep R and L small. Table 3 collects the governing constants.

2.8 Pruning unreachable regions

Any "explore until everything is seen" criterion has a subtle flaw. Some unknown cells can never be seen, such as the interiors of walls or pockets sealed off by obstacles. The map therefore runs a periodic morphological pass. It labels the connected components of unknown space, and for each component that no free cell touches, and that is larger than a minimum size, it relabels the component as an obstacle. Reachability is tested by dilating the component and intersecting it with the free mask.

This turns "unknown and unreachable" into "known to be solid." It lets the exploration score reach its threshold, and it stops frontier detection from forever flagging the boundary of an enclosed void. It is a small but characteristic example of using image-processing primitives, namely connected components, dilation and distance transforms, to reason about the topology of free space rather than about occupancy cell by cell.

Parameter	Value	Role
resolution	8 px/cell	world-to-grid downsampling (64× fewer cells)
every_n	3	lidar ray sub-sampling for speed
empty_zone_value	−0.602	free-evidence increment ($\approx \log_{10} \frac{1}{4}$)
obstacle_zone_value	+2.0	obstacle-evidence increment
clip range	[−40, 40]	bounded belief, for responsiveness
lidar_noise_std σ	2.5	assumed range-noise scale
lidar_obstacle_margin	4σ	guard band rejecting max-range no-echo returns
max_lidar_dist_clip	40	range-proportional free-marking setback

Table 3: Mapping parameters. The thresholds for the ternary map sit at zero.

2.9 Trade-offs

The additive model is fast and merges trivially, but it is not probabilistically calibrated. The effective value is an evidence score, not a posterior probability, so it cannot be thresholded by a principled risk bound. The eight-to-one resolution hides sub-drone geometry, which the planner compensates for by inflating obstacles (Section 4). And the complex-channel scheme, elegant as it is, models confidence as a binary GPS-or-not flag rather than as a continuous uncertainty. A fuller treatment would weight evidence by the estimated pose covariance from the EKF.

Score d'exploration

$$\frac{\# \text{ cases libres ou obstruées}}{\# \text{ cases de la cartographie}}$$

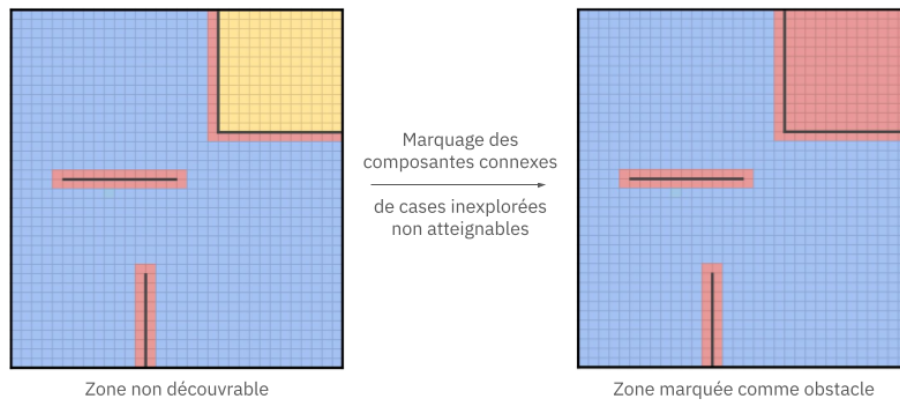


Figure 6: Pruning unreachable space for the exploration score. Connected components of unexplored cells that no free cell can reach (left) are relabeled as obstacles (right), so the fraction-explored score can complete instead of stalling on an enclosed void. (Slide from the team presentation; labels in French.)

3 Decentralized exploration and task allocation

With a map in hand, the swarm must decide where to look next. This section describes how the unexplored frontier of the map is detected and turned into a small set of candidate targets, and how those targets are then shared out among the drones as a formal assignment problem.

3.1 Problem and constraints

Exploration must be fast, because efficiency is scored. It must be complete, because exploration percentage is scored. And it must be coordinated, because ten drones piling onto the same corridor waste the swarm. It must achieve this without a central planner and without a global view, since each drone knows only what it has seen and what has reached it through local communication. The chosen strategy is classical frontier-based exploration, repeatedly driving toward the boundary between known-free and unknown space, augmented with a genuine combinatorial assignment step so that the swarm spreads out.

3.2 Detecting frontiers as occupancy edges

A frontier cell is a free cell adjacent to unknown space. On the ternary map this is exactly an edge between the FREE and UNDISCOVERED labels, which differ by two in the integer encoding. Detecting all frontier cells is therefore a single finite-difference pass:

$$\mathcal{F} = \{ \mathbf{c} : |M(\mathbf{c}) - M(\mathbf{c}')| = 2, \mathbf{c}' \text{ a 4-neighbor of } \mathbf{c} \}, \quad (9)$$

computed with array differences along both axes and combined with a logical OR. This is vectorized edge detection rather than a per-cell scan.

Exploration par frontière

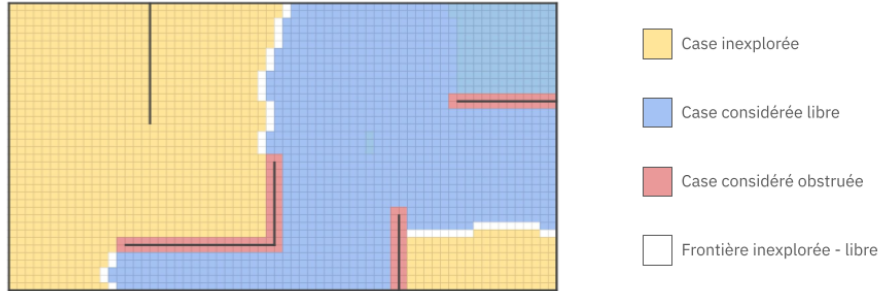


Figure 7: Frontier detection on the ternary map. The frontier cells (white) are exactly the free cells (blue) that border unexplored space (yellow), found as the edge between the two labels. (Slide from the team presentation; labels in French.)

3.3 From cells to targets: clustering and arc splitting

Raw frontier cells are grouped into coherent frontier objects with **DBSCAN**, using a neighborhood radius of `clustering_epsilon = 2` cells and a minimum cluster size of `min_frontier_size = 3`. DBSCAN suits the task well. It finds connected runs of frontier cells without being told how

many there are, and it discards isolated specks as noise. Each surviving cluster becomes a frontier carrying its cells and a lazily-computed centroid.

A second, geometry-aware refinement handles a failure mode of pure clustering. When the drone sits inside a partially-mapped room, the entire surrounding boundary can form one large arc whose centroid lies in already-explored space. Driving to that centroid would be useless. The controller detects this by computing the angular positions of a cluster’s cells about its centroid, sorting them, and measuring the largest angular gap:

$$\text{is_large_arc} \iff \max_k (\theta_{(k+1)} - \theta_{(k)}) < \text{frontier_split_threshold} = 2 \text{ rad.} \quad (10)$$

A small maximal gap means the cells wrap most of the way around the centroid, that is an arc greater than 180 degrees. Such a frontier is split in two with k-means, recursively, until each piece is a well-behaved patch with a reachable centroid.

Répartition des frontières

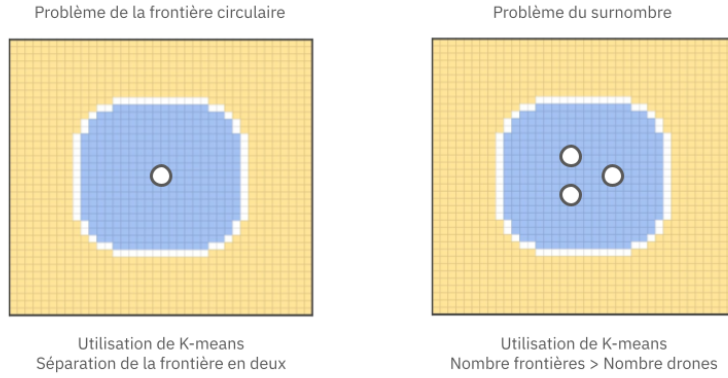


Figure 8: Two failure modes resolved by k-means splitting. A circular frontier whose centroid falls in already-explored space (left), and the case of fewer frontiers than drones (right), are both fixed by splitting the offending frontier into smaller, well-behaved pieces. (Slide from the team presentation; labels in French.)

3.4 Target assignment as a linear assignment problem

The coordination core is the assignment of frontiers to drones, and it is framed as a classical operations-research problem. Given the drone’s view of where its peers are, it builds a cost matrix $C \in \mathbb{R}^{m \times n}$ over m drones and n frontiers:

$$C_{ij} = \frac{\|\mathbf{p}_i - \bar{\mathbf{f}}_j\|}{|\mathcal{F}_j| + 1} \cdot \begin{cases} \text{los_multiplier} = 0.5 & \text{if } \bar{\mathbf{f}}_j \text{ is in line of sight of drone } i, \\ 1 & \text{otherwise,} \end{cases} \quad (11)$$

where $\bar{\mathbf{f}}_j$ is frontier j ’s centroid and $|\mathcal{F}_j|$ its size. Three modeling choices live in this one expression. Dividing by frontier size biases the swarm toward large frontiers, which carry more unexplored space per visit and resist being abandoned for a nearby sliver. The line-of-sight discount favors frontiers the drone can reach in a near-straight line over ones hidden behind walls. And Euclidean distance is used deliberately as a cheap surrogate for true path length, since computing an A* distance for every drone-frontier pair every cycle would be far too expensive.

The matrix is then solved optimally with the **Hungarian algorithm**, available as SciPy’s `linear_sum_assignment`, which returns the minimum-cost one-to-one matching in $O(\max(m, n)^3)$. Two practical wrinkles complete the scheme. If there are fewer frontiers than drones, the largest frontier is repeatedly split until $n \geq m$, so the assignment is not forced to leave drones idle. And because the matching is computed over all known drones but each drone obeys only its own row, a drone that the optimal matching happens to leave unassigned falls back to its individually cheapest frontier. Algorithm 3 states the full procedure.

Algorithm 3: Frontier assignment (per drone, per decision)

Input: local frontier set $\mathcal{F}$, self position $\mathbf{p}_0$, known peer positions

Output: a frontier to head toward

```

1 if  $\mathcal{F} = \emptyset$  then return none
2  $P \leftarrow [\mathbf{p}_0, \text{known peer positions}]; \quad m \leftarrow |P|$ 
3 while  $|\mathcal{F}| < m$  do
4   | split the largest frontier with k-means
5   | if the split produced nothing then break
6 for each drone  $i$  and frontier  $j$  do
7   |  $C_{ij} \leftarrow \frac{\|\mathbf{p}_i - \bar{\mathbf{f}}_j\|}{|\mathcal{F}_j| + 1} \times (0.5 \text{ if line-of-sight else } 1)$ 
8  $(\text{rows}, \text{cols}) \leftarrow \text{HUNGARIAN}(C)$ 
9 if self is left unassigned then return  $\arg \min_j C_{0j}$ 
10 return the frontier assigned to self

```

A worked micro-example. Suppose a drone knows of two peers, and the map currently offers two frontiers: a small one nearby, with $|\mathcal{F}_1| = 4$, and a large one farther off, with $|\mathcal{F}_2| = 40$. Even though frontier 1 is closer, the size normalization $1/(|\mathcal{F}| + 1)$ can make the large frontier cheaper. The solver may then send the nearby drone to the big frontier and dispatch a farther peer to mop up the small one, which is the swarm-level outcome of covering more unknown area sooner rather than the locally greedy one. A line-of-sight discount on either entry can flip the assignment again. The point is that all three considerations are weighed jointly by the optimizer instead of through hand-written conditionals.

Complexity. The pieces compose cheaply. Frontier-cell detection is one vectorized pass, $O(\text{cells})$. DBSCAN is roughly $O(f \log f)$ in the number of frontier cells f . The cost matrix is $O(mn)$ to fill, each entry needing one line-of-sight check, and the Hungarian solve is $O(\max(m, n)^3)$. With at most ten drones and a handful of frontiers these are all small constants in practice, which is why the optimal cubic assignment is affordable every decision cycle.

3.5 Solving the assignment locally

Each drone solves the assignment on its own, using only the peer positions it actually knows. Those are the positions of the teammates it is currently in contact with, since a drone hears only from peers within communication range and only keeps recent reports. Drones that are out of range simply do not appear in the cost matrix.

The reason for solving the problem this way is pragmatic rather than theoretical. A fully coordinated, provably-optimal distributed assignment would require consensus rounds and a shared, synchronized view of the swarm, which is expensive and fragile. Solving the assignment locally, from whatever neighbors happen to be in range, gives satisfying results in practice and keeps the implementation simple. When neighbors are in contact their independent solutions

Répartition des frontières

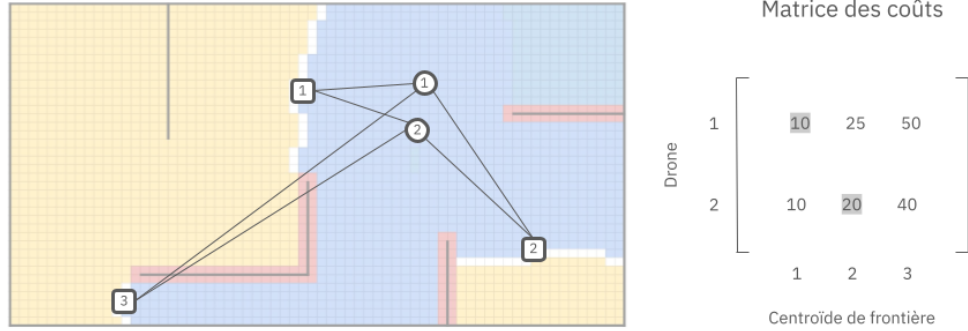


Figure 9: Frontier assignment as a cost-matrix problem. Each drone-to-frontier-centroid pair carries a cost (right); the Hungarian algorithm selects the minimum-cost one-to-one matching (shaded entries), spreading the swarm across distinct frontiers. (Slide from the team presentation; labels in French.)

tend to agree and the swarm fans out. When they are not, the worst case is a transient overlap that resolves itself as soon as the drones come back into range and exchange positions again.

3.6 Knowing when to stop

Exploration is not open-ended. An exploration score, defined as the fraction of cells that hold any evidence, is computed by counting non-zero grid cells, after first relabeling provably-unreachable pockets of unknown space as obstacles (Section 2). When the score crosses `min_exploration_score = 0.9`, the state machine moves into its end-of-mission phase, in which a subset of drones revisits earlier wounded sightings to confirm rescues. Table 4 lists the parameters of this subsystem.

Parameter	Value	Role
<code>clustering_epsilon</code>	2 cells	DBSCAN neighborhood radius
<code>min_frontier_size</code>	3 cells	minimum cells for a frontier, and DBSCAN min samples
<code>frontier_split_threshold</code>	2 rad	max angular gap below which an arc is split
<code>los_multiplier</code>	0.5	cost discount for frontiers in line of sight
<code>min_exploration_score</code>	0.9	explored fraction that ends the exploration phase

Table 4: Exploration and assignment parameters.

4 Motion planning and reactive control

Choosing a goal is only half the problem. The drone still has to reach it without harming itself. This section presents the two-layer stack that does so, from the cost map and the global plan down to the controller that follows the plan and the reactive layer that keeps it out of trouble.

4.1 Problem and constraints

Once a state has chosen a goal, whether a frontier centroid, the rescue center or a wounded person, the drone must get there safely. Safety is load-bearing here. Collisions cost health, which is scored, and a drone carrying a wounded person should hug obstacles even less than usual. The motion stack is therefore two-layered. A global planner produces a path that is optimal with respect to a cost map, and a reactive controller follows it while continuously bending away from obstacles the planner could not resolve at grid resolution.

4.2 The cost map: turning a binary map into a risk field

The planner does not search the occupancy grid directly. It first builds a real-valued cost map in which every cell carries a traversal cost. Free cells get a base cost of `constant_cost` = 1. Obstacle cells, and undiscovered cells, get infinite cost, so the planner refuses to route through the unknown. On top of this, every obstacle is surrounded by a decaying penalty computed from an exact Euclidean **distance transform** $D(\mathbf{c})$, the distance from each free cell to the nearest obstacle:

$$\text{cost}(\mathbf{c}) = 1 + \begin{cases} \frac{\text{obstacle_base_penalty}}{(D(\mathbf{c}) + 0.01)^2} & D(\mathbf{c}) \leq \rho, \\ 0 & \text{otherwise,} \end{cases} \quad (12)$$

with `obstacle_base_penalty` = 1000 and an inflation radius $\rho = 6$ cells, chosen as three drone radii in world units divided by the resolution. The inverse-square falloff makes the penalty rise steeply close to a wall but become negligible a few cells away (Figure 10, left). The immediate consequence is that, inside a corridor, the penalties from the two walls add up to a cost valley along the centerline (Figure 10, right). The planner therefore prefers to travel down the middle of corridors without tight passages being forbidden outright.

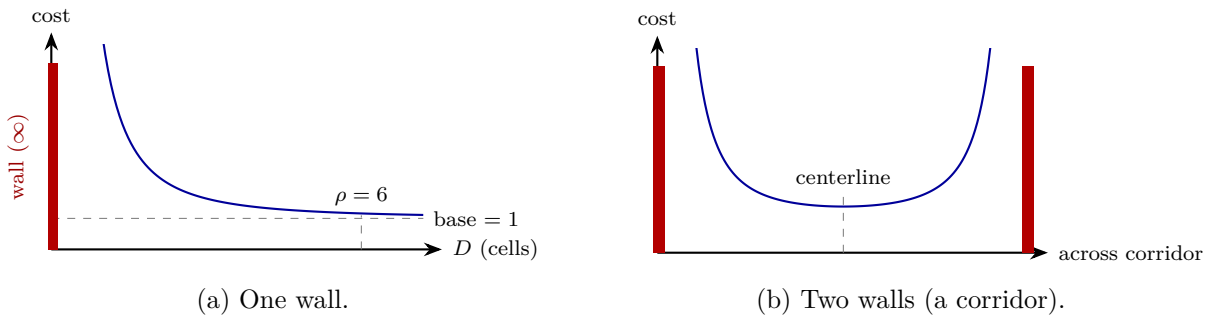


Figure 10: Cost-map profile from Equation (12). A single wall (left) pushes paths away with a steep inverse-square penalty inside $\rho = 6$ cells. Between two walls (right) the penalties add, creating a cost valley that draws optimal paths toward the middle of the corridor.

4.3 A* on an 8-connected grid

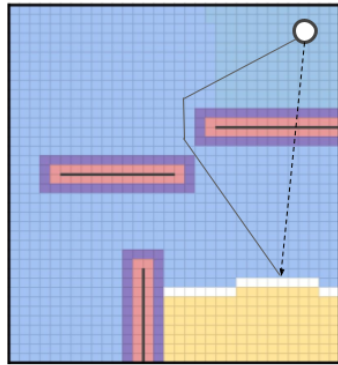
The path is found by **A*** over the cost map with 8-connectivity. Straight moves cost one times the cell cost, diagonal moves cost $\sqrt{2}$ times, and the heuristic is the octile distance:

$$h(\mathbf{a}, \mathbf{b}) = (|\Delta x| + |\Delta y|) + (\sqrt{2} - 2) \min(|\Delta x|, |\Delta y|). \quad (13)$$

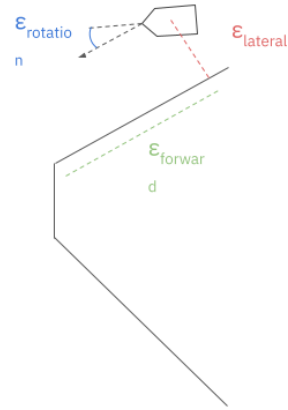
This is the exact length of an obstacle-free 8-connected path, and it is therefore admissible and consistent. A* returns a least-cost path, and because the cost map encodes obstacle risk, least-cost means reasonably short and away from walls. Consistency matters in practice as well as in theory. With a consistent heuristic A* never needs to reopen a closed node, so the search expands each cell at most once and runs in $O(N \log N)$ in the number of reachable free cells N . The infinite cost on undiscovered and obstacle cells prunes most of the grid before the search even begins. Start and goal positions are first snapped to the nearest free cell by a small outward spiral, so a goal that lands on an inflated or unknown cell still yields a plan rather than failing.

Calcul et suivi de chemins

Utilisation d'A* pour le calcul des chemins



Heuristique : ligne droite
Inflation douce des obstacles
Simplification du chemin (LOS / Bresenham)



Asservissement sur un segment

Figure 11: Global planning and path following. Left: A* over the inflated cost map (obstacle halos in purple), with a straight-line heuristic and line-of-sight/Bresenham simplification. Right: the segment-tracking errors, rotation, lateral and forward, each regulated by its own PID loop. (Slide from the team presentation; labels in French.)

4.4 Path simplification, with context-dependent caution

A raw grid path is a long staircase of unit steps. Feeding it directly to the follower would produce jittery motion, so the path is simplified in two stages. The first stage keeps only the vertices where the step direction changes, collapsing straight runs to their endpoints. The second stage greedily removes intermediate waypoints. From the current kept waypoint it looks as far ahead as possible to a waypoint reachable by a straight segment whose every cell stays below a cost allowance, and jumps there.

The allowance is where caution becomes contextual. A free-flying drone uses a generous allowance of `shortcut_penalty_allowance = 250`, accepting near-wall shortcuts to save time. A drone carrying a wounded person uses `caution_penalty_allowance = 1`, which permits shortcuts only through essentially zero-risk cells and thus keeps the precious cargo well clear of walls. The same A* path is post-processed differently depending on what is at stake. It is a neat example of letting the mission state reshape a generic algorithm.

4.5 Following the path: decoupled pure pursuit

The follower turns a polyline into commands using three independent PID controllers, each acting on a geometric component of the tracking error. Let $\hat{\mathbf{u}}$ be the unit vector of the current

segment and $\mathbf{v}$ the vector from the segment's start to the drone. The follower computes a heading error, a cross-track (lateral) error, and an along-track (longitudinal) error (Figure 12):

$$e_\theta = \text{atan2}(u_y, u_x) - \theta, \quad e_\perp = -(\hat{\mathbf{u}} \times \mathbf{v}), \quad e_\parallel = \|\text{segment}\| - \hat{\mathbf{u}} \cdot \mathbf{v}. \quad (14)$$

The cross-track error is the signed perpendicular distance to the segment, and the along-track error is the remaining distance to the next waypoint. Each error drives a PID law, $u = \text{clip}(k_p e + k_d \dot{e} + k_i \int e, -1, 1)$, with mode-specific derivative terms. The rotation loop reads angular rate straight from the odometer, and the lateral loop reconstructs lateral velocity from odometry.

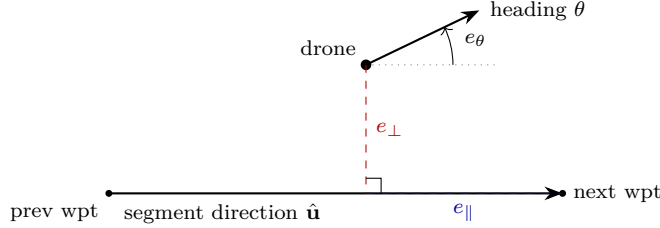


Figure 12: Tracking geometry. The drone's error relative to the active segment splits into a heading error e_θ , a cross-track component $e_\perp$ and an along-track component $e_\parallel$, each regulated by its own PID loop.

A small but consequential gating rule decouples the loops. The lateral and forward commands are issued only when the heading error is small, that is when $|e_\theta| < \pi/15$. The drone thus turns to face roughly where it is going before it translates, which keeps the linear-error geometry of Equation (14) valid and stops the drone from sliding sideways into walls while mis-oriented.

The gains in Table 5 tell their own story. Every loop runs with $k_i = 0$, so there is no integral action anywhere. The forward loop is strongly over-damped, with $k_d = 11 \gg k_p = 1.6$, which trades responsiveness for a near-monotone approach to each waypoint. That is desirable when overshooting means hitting a wall. The lateral loop is likewise damping-dominant, keeping cross-track corrections smooth, while the heading loop is the only lightly-damped one, so the drone snaps to a new heading quickly and then lets the gentler translational loops take over. Omitting integral action is defensible here precisely because there is no persistent bias to integrate away. The dominant disturbance is the obstacle field, which the reactive layer below handles directly.

Loop	k_p	k_d	k_i	Character
Heading (rotation)	$9/\pi \approx 2.86$	$k_p/10 \approx 0.29$	0	lightly damped, fast
Cross-track (lateral)	0.311	1.367	0	damping-dominant, smooth
Along-track (forward)	1.6	11.0	0	strongly over-damped, no overshoot

Table 5: PID gains for the three decoupled tracking loops. No loop uses integral action.

4.6 The reactive layer: potential-field offsets

The planned path is computed from the occupancy grid, which is imperfect. The grid can be wrong because the sensors that built it are noisy, and it cannot represent moving obstacles such as other drones at all. To close that gap, the follower adds a **potential-field** correction computed directly from the live lidar. The main purpose of this correction is to react to exactly those two discrepancies: mapping inaccuracy from noisy sensors, and moving obstacles, above all other drones, that the static map never captures.

Every lidar ray closer than $\text{max_inflation_obstacle} = 20$ produces a repulsion proportional to how close it is. Rays are signed by hemisphere, so obstacles on the left push the drone right and obstacles on the right push it left, while obstacles ahead push it back and obstacles behind push it forward:

$$\text{offset} = \text{mean}_{d_k < 20} [(20 - d_k) \cdot s_k], \quad s_k \in \{-1, +1\} \text{ by hemisphere.} \quad (15)$$

These offsets are simply added to the cross-track and along-track errors before the PID step, so avoidance and tracking are regulated by the same loops. The drone is continuously nudged off the planned line by nearby obstacles and relaxes back onto it once they clear. Figure 13 shows how the deliberative and reactive layers compose.

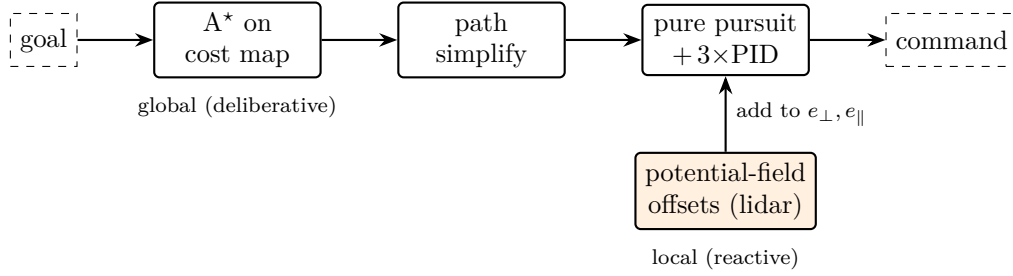


Figure 13: The two-layer motion stack. A deliberative global plan sets the route, and a reactive potential field, recomputed every step from raw lidar, perturbs the tracking errors to avoid what the plan could not see.

Two situations get extra care, both aimed at protecting drone health. The first is a genuinely blocked route. Rather than reacting to every nearby obstacle, the follower checks only a narrow cone of lidar rays pointing toward the next waypoint, and declares the route blocked only if the nearest return in that cone is both closer than the waypoint and within a hard threshold. When that happens it asks the planner for a new path. The drone thus replans when its intended way is truly sealed, but ignores obstacles it is already steering around. The second situation is a close encounter with a teammate. Because all ten drones run the same controller, they tend to converge on the same corridors, and drone-to-drone collisions cost health just like wall collisions. When the semantic sensor reports a teammate nearby, the forward command is recomputed as a cap on closing speed rather than as a position error, which defuses the encounter without any explicit negotiation.

4.7 Trade-offs

The two-layer split is the classic hybrid-architecture bargain. The planner gives globally sensible routes but cannot react in real time, while the potential field reacts instantly but is myopic and can stall in local minima. Placing the reactive correction inside the path follower, rather than as a separate arbiter, keeps the two from fighting in the common case while preserving the planner’s global guidance. The absence of integral action means a sustained disturbance is not actively rejected, but in this environment there is no such disturbance to reject, and the reactive layer covers the transient ones.

Calcul et suivi de chemins

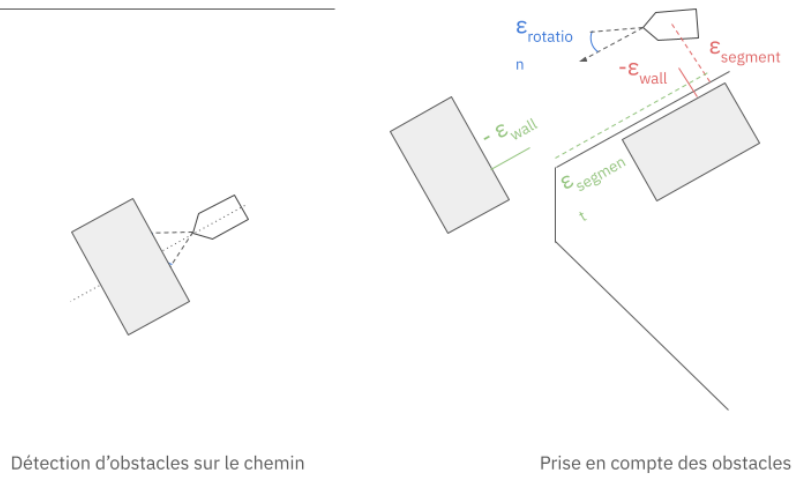


Figure 14: The reactive layer in action. Obstacles detected on the path (left) inject potential-field offsets, a wall repulsion ϵ_{wall} and a segment-pull $\epsilon_{\text{segment}}$, into the tracking errors (right), bending the drone around hazards the static plan could not resolve. (Slide from the team presentation; labels in French.)

5 Communication mechanisms under uncertainty

Everything so far has described a single drone. This section is about what turns ten of them into a swarm: the messages they exchange, and the care taken to keep that shared picture coherent despite noise, motion and intermittent contact.

5.1 Problem and constraints

A swarm is only as good as its shared situational awareness, and that awareness must be assembled from noisy, partial, asynchronous observations. Three problems define this subsystem. The first is persistence. A fact discovered by one drone, such as a wounded person’s location or a region of the map, must survive and spread, even to drones that never come near the discoverer. The second is disambiguation. The same physical object is observed many times, by many drones, each observation displaced by sensor noise and by the object’s own motion, and the swarm must not treat these as many different objects. The third is mutual exclusion. Two drones must not both commit to rescuing the same person.

A drone communicates only with teammates within 250 pixels, and messages flow once per timestep. The design is also written as though bandwidth were scarce, even though the simulator does not bill for it, both for realism and because unbounded shared state would otherwise grow without limit.

5.2 Three channels

Each living drone broadcasts three message types per step, summarized in Table 6. The first carries its own identity, position, state and timestamp, together with everything it has heard about other drones. The second carries its full occupancy grid. The third carries the positions of any wounded it has seen this step. The receiver dispatches each message to a dedicated merge routine. The interesting engineering is in how each merge copes with uncertainty. Internally a receiver keeps a few dictionaries keyed by drone identity, holding the last-known information per peer and the last time a map was merged with that peer, so that merging, recency checks and throttling are all constant-time lookups.

Channel	Payload	Merge rule and uncertainty handling
swarm_info	identity, position, state, timestamp (own and relayed peers)	keep the newest entry per drone (gossip); act only on entries within <code>max_info_delay= 2</code> steps
mapping	full complex occupancy grid	elementwise average, throttled to once per <code>time_interval= 30</code> steps per peer
wounded_sighting	positions seen this step	absorb into the nearest existing sighting within <code>same_wounded_radius= 100</code>

Table 6: The three broadcast channels and how each reconciles uncertain, redundant data.

5.3 Persistent information that propagates without contact

The swarm-information channel implements what is effectively an epidemic, or gossip, protocol. Because a drone rebroadcasts not only its own state but its entire table of last-known peer states, a fact hops outward through the network. Drone A tells B, and later B tells C, even though A and C are never within 250 pixels of each other at the same time. The merge keeps, for each drone, only the entry with the newest timestamp, which makes the process convergent and loop-free, since re-hearing an old copy of a fact never overwrites a newer one.

A separate recency gate then decides which facts are fresh enough to act on. Peer positions used for frontier assignment, and the wounded locks described below, are drawn only from entries seen within the last $\text{max_info_delay} = 2$ steps. The swarm therefore plans on live data while still remembering stale data for the sole purpose of propagating it. The distinction between retaining information and trusting it is the core idea of this channel.

Communication

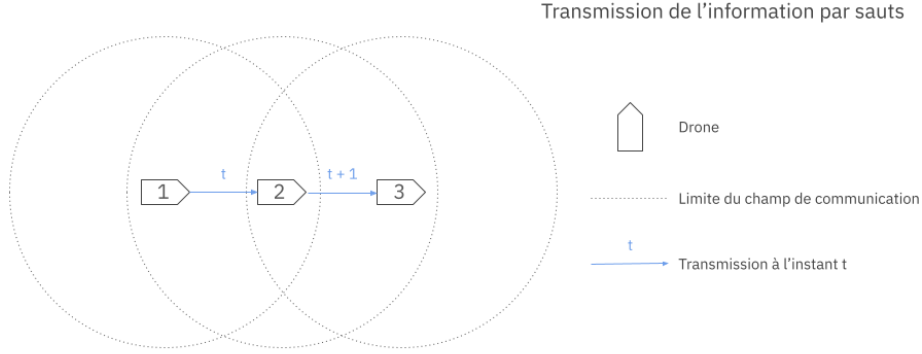


Figure 15: Gossip propagation by hops. Drone 1 passes information to drone 2 at step t , and drone 2 relays it onward to drone 3 at step $t + 1$, so a fact spreads across the swarm even between drones whose communication ranges never overlap. (Slide from the team presentation; labels in French.)

5.4 Disambiguating noisy, repeated observations

The wounded-sighting channel must answer a clustering question. Is this report a new victim, or one we already know, given that positions scatter under noise? The solution is deduplication by spatial proximity. A reported sighting is registered as new only if it is farther than $\text{same_wounded_radius} = 100$ from every sighting already on the list, as illustrated in Figure 16.

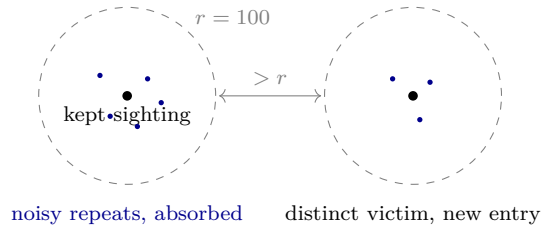


Figure 16: Proximity deduplication. Repeated noisy observations of one victim fall inside a single radius and collapse to one entry, while a genuinely distinct victim farther than r away starts a new one. The radius is chosen larger than the localization noise but smaller than the expected victim spacing.

This is, in effect, single-pass greedy clustering with a fixed radius. The kept sightings act as cluster representatives, and any observation within one radius is absorbed. The radius is chosen larger than the expected localization error, so the noisy cloud of repeated detections of one victim collapses to a single entry, yet smaller than the expected spacing between distinct victims, so different people stay separate. The same mechanism, with its own radius, deduplicates kill-zone

sightings. A complementary filter rejects spurious sightings near the rescue center, or on cells already known to be free, because a sighting there usually comes from a drone dropping off a victim rather than from a new one. Finally, an occlusion test sweeps a cone of lidar rays toward the reported bearing and discards the sighting if any ray returns an obstacle closer than the victim, so a drone does not chase a person it has merely glimpsed through a gap in a wall.

Relevé des positions des blessés

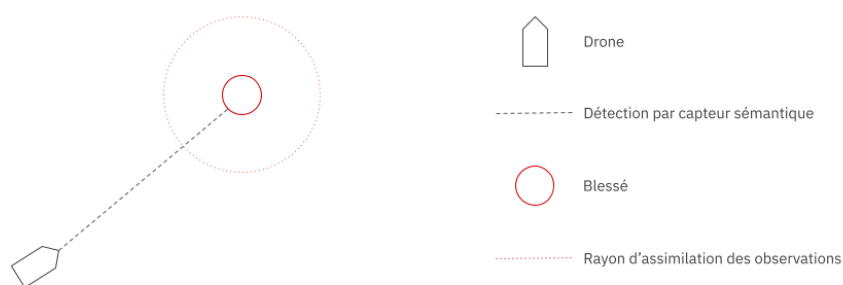


Figure 17: Recording a wounded sighting. The semantic sensor detects a victim at a bearing and range; any later observation falling inside the assimilation radius (dotted) is absorbed into the same entry rather than logged as a new person. (Slide from the team presentation; labels in French.)

5.5 Mutual exclusion: wounded locking

To stop two drones converging on one victim, the swarm uses an implicit, soft lock. Any drone whose state is grasping a wounded person, or carrying one back, is by virtue of broadcasting that state advertising that it has claimed a victim near its position. Every other drone, before committing to a detected victim, filters out any detection within `wounded_conflict_threshold = 20` of a locked drone's position.

The lock needs no dedicated protocol or acknowledgement. It is inferred from the state that is already being gossiped, and it expires automatically through the same recency gate when the holder stops advertising. This reuse of existing state, rather than a new handshake, is what keeps the mechanism simple and robust.

5.6 End-of-mission coordination

The same gossiped-state idea also drives the end-of-mission phase. Once exploration is sufficiently complete, only a fixed subset of drones, the lowest-indexed two, take on the job of revisiting recorded wounded sightings to confirm rescues. Each of them skips sightings that are too close to the rescue center, or that no longer sit near an obstacle, since those are likely stale entries left by drop-offs or by victims that have already been collected.

Assigning this task by drone index needs no election and no communication at all. It is a deterministic, zero-overhead way to divide a verification workload that would be wasteful to replicate across all ten drones.

5.7 Fusing maps of mixed confidence

Map sharing reuses the complex grid of Section 2. To bound redundant work, a pair of drones merges maps at most once every $\text{time_interval} = 30$ steps, and when they do, the merge is a single elementwise average, $G \leftarrow \frac{1}{2}(G + G_{\text{other}})$. Because the grid is complex, this one operation fuses both confidence channels at once and keeps them separate. GPS evidence averages with GPS evidence in the real part, noisy with noisy in the imaginary part, and the precedence rule of Equation (7) still prefers GPS evidence after the merge. This is exactly the payoff of the complex-channel representation: mixed-confidence fusion for the price of an average.

The throttling deserves a word on scalability. In principle every drone could merge every peer’s full map every step, but with ten drones that is dozens of grid-sized operations per step, most of them redundant, since a peer’s map changes little between consecutive steps. Capping merges to once per thirty steps per pair turns a quadratic-per-step cost into something negligible while losing almost no information. On a real radio link this kind of restraint would be mandatory rather than optional, and the same philosophy of designing as though the resource were scarce runs through the timestamping and deduplication on the other two channels.

6 Conclusion

The four subsystems are not independent modules bolted together. They are layers of a single idea, which is to maintain a shared, confidence-aware world model and to act on it greedily but coordinately. The complex-valued occupancy grid is the substrate. It accumulates noisy evidence into a bounded belief and keeps trustworthy and untrustworthy evidence separable. Exploration reads that substrate to find frontiers and distributes them across the fleet by solving a linear assignment problem locally, so the swarm fans out without a central authority. The motion stack turns each chosen goal into safe motion by planning on a risk-inflated cost map and following the plan with a reactive controller that trusts live lidar over the imperfect map. And the communication layer is what makes the first three collective. It propagates persistent facts across drones that never meet, collapses the swarm’s noisy repeated observations into a coherent picture, and arbitrates who rescues whom, while always treating retained information and trusted information as different things.

On simple maps with only one drone, the solution yields very satisfying results, with scores up to 97%, meaning the drone doesn’t lose any health, explores the whole map and rescues every wounded person.

On more complex maps, with several drones and difficulty zones (no-gps, no-communication, kill-zones), the solution proves to be quite resilient, even though drones frequently collide and every aspect of the solution deteriorates (inaccurate mapping in no-gps zones ...).

Finally, this solution placed 3rd in the Swarm-Rescue challenge, a testimony to its quality.